ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

BLM4522 - AĞ TABANLI PARALEL DAĞITIM SİSTEMLERİ

PROJE - 3

Veritabanı Güvenliği ve Erişim Kontrolü

Şevval Ensarioğlu - 22290742

GitHub Linki:

https://github.com/SevvalEnsarioglu/blm4522-proje3-db-security

İçindekiler

1 Giriş **3**

1.1 Projenin Amacı 3

1.2 Kullanılan Araçlar ve Teknolojiler 3

1.3 MSSQL ile PostgreSQL Karşılaştırması 3

2 Erişim Yönetimi **5**

2.1 Mevcut Rollerin İncelenmesi 5

2.2 Yeni Güvenlik Rollerinin Oluşturulması 5

2.3 Yetki Testi — Erişim Engelleme 6

2.4 REVOKE ile Yetki Geri Alma 6

2.5 pg_hba.conf — Host Tabanlı Erişim Kontrolü 7

3 Veri Şifreleme **8**

3.1 pgcrypto Extension'ının Kurulumu 8

3.2 Hassas Veri Tablosunun Oluşturulması 8

3.3 Veri Şifreleme — pgp_sym_encrypt 8

3.4 Şifreli Verinin Görüntülenmesi 9

3.5 Şifre Çözme — pgp_sym_decrypt 9

3.6 Yanlış Anahtar Testi 9

4 SQL Injection Testleri **11**

4.1 Savunmasız Senaryo Simülasyonu 11

4.2 Korumalı Yöntem — Prepared Statements 11

4.3 Ek Güvenlik — VIEW ile Veri Maskeleme 12

5 Audit Logları **14**

5.1 Audit Log Tablosunun Oluşturulması 14

5.2 Audit Trigger Fonksiyonunun Oluşturulması 14

5.3 Trigger'ların Tablolara Bağlanması 15

5.4 Audit Sisteminin Test Edilmesi 16

5.5 Audit Loglarının Sorgulanması 16

5.6 Audit Log Özet Raporu 16

6 Sonuç ve Değerlendirme **17**

6.1 Yapılan Çalışmaların Özeti 17

6.2 Öğrenilen Dersler 17

6.3 Kaynaklar 18

1 Giriş

Bu proje, BLM4522 Ağ Tabanlı Paralel Dağıtım Sistemleri dersi kapsamında hazırlanmıştır. Proje kapsamında PostgreSQL veritabanı yönetim sistemi üzerinde güvenlik analizi ve erişim kontrolü çalışmaları gerçekleştirilmiştir.

Çalışmada **DVD Rental (Pagila)** örnek veritabanı kullanılmıştır. Bu veritabanı; müşteri bilgileri, ödeme kayıtları ve kişisel veriler içermesi nedeniyle güvenlik çalışmaları için gerçekçi bir ortam sunmaktadır.

1.1 Projenin Amacı

- Kullanıcı erişim yetkilerini rol tabanlı sistemle yönetmek
- Hassas verileri şifreleyerek yetkisiz erişime karşı korumak
- SQL Injection saldırılarını simüle etmek ve savunma yöntemlerini uygulamak
- Audit log sistemi kurarak kullanıcı aktivitelerini izlemek

1.2 Kullanılan Araçlar ve Teknolojiler

Araç	Sürüm	Kullanım Amacı
PostgreSQL	17	Ana veritabanı yönetim sistemi
DBeaver	25.2.4	Veritabanı yönetim arayüzü
pgcrypto	—	Sütun düzeyinde veri şifreleme
pgaudit	—	Audit log ve aktivite izleme
DVD Rental DB	—	Örnek veritabanı (15 tablo, 16.000+ kayıt)

Tablo 1: Kullanılan araçlar ve teknolojiler

1.3 MSSQL ile PostgreSQL Karşılaştırması

Proje tanımında MSSQL terimleri kullanılmıştır. Aşağıdaki tablo, bu terimlerin PostgreSQL karşılıklarını göstermektedir:

MSSQL Kavramı	PostgreSQL Karşılığı
SQL Server Authentication	`LOGIN ROLE` + MD5 şifre
Windows Authentication	`pg_hba.conf` host tabanlı erişim
TDE (Transparent Data Encryption)	`pgcrypto` sütun şifreleme
SQL Server Audit	`pgaudit` + Trigger tabanlı audit

Tablo 2: MSSQL ve PostgreSQL güvenlik kavramları karşılaştırması

2 Erişim Yönetimi

PostgreSQL'de erişim yönetimi **Rol Tabanlı Erişim Kontrolü (RBAC)** ile sağlanmaktadır. Bu yapı, MSSQL'deki SQL Server Authentication ve Windows Authentication mekanizmalarının işlevsel karşılığıdır.

2.1 Mevcut Rollerin İncelenmesi

Sistemdeki mevcut roller ve yetkileri incelenmiştir:

```
SELECT rolname, rolcanlogin, rolsuper, rolcreatedb
FROM pg_roles
WHERE rolname NOT LIKE 'pg_%'
ORDER BY rolname;
```

2.2 Yeni Güvenlik Rollerinin Oluşturulması

Üç farklı güvenlik düzeyinde rol oluşturulmuştur:

```
-- 1. Sadece okuma yetkisi olan analist
CREATE ROLE analist LOGIN PASSWORD 'analist123';
GRANT CONNECT ON DATABASE dvdrental TO analist;
GRANT USAGE ON SCHEMA public TO analist;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO analist;

-- 2. Belirli tablolara yazma yetkisi olan operat r
CREATE ROLE operator LOGIN PASSWORD 'operator123';
GRANT CONNECT ON DATABASE dvdrental TO operator;
GRANT USAGE ON SCHEMA public TO operator;
GRANT SELECT, INSERT, UPDATE ON payment, rental TO operator;

-- 3. Tam yetkili veri y neticisi
CREATE ROLE veri_yoneticisi LOGIN PASSWORD 'yonetici123';
GRANT ALL PRIVILEGES ON DATABASE dvdrental TO veri_yoneticisi;
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO
   veri_yoneticisi;
```

Rol	Yetki Düzeyi	İzin Verilen İşlemler
analist	Sadece okuma	SELECT — tüm tablolarda
operator	Okuma + Yazma	SELECT, INSERT, UPDATE — payment, rental
veri_yoneticisi	Tam yetki	Tüm tablolarda tüm işlemler

Tablo 3: Oluşturulan roller ve yetki düzeyleri

2.3 Yetki Testi — Erişim Engelleme

Analist rolünün yetkisiz işlem yapması engellenmiştir:

```
-- Analist rol ne ge
SET ROLE analist;

-- Veri eklemeye al HATA vermeli
INSERT INTO film(title) VALUES ('test');
-- SONU : ERROR: permission denied for table film

-- Rol s f rla
RESET ROLE;
```

Sonuç: `ERROR: permission denied for table film` hatası alınmıştır. Erişim kontrolü başarıyla çalışmaktadır.

2.4 REVOKE ile Yetki Geri Alma

```
-- Analistten m teri tablosu eri imini kald r
REVOKE SELECT ON customer FROM analist;

-- Test et
SET ROLE analist;
SELECT * FROM customer LIMIT 5;
-- SONU : ERROR: permission denied for table customer
```

```sql
RESET ROLE;
```

2.5 pg_hba.conf — Host Tabanlı Erişim Kontrolü

MSSQL'deki Windows Authentication'ın PostgreSQL karşılığı olan `pg_hba.conf` dosyası ile hangi kullanıcının hangi IP adresinden bağlanabileceği kontrol edilmektedir:

```sql
-- Dosya konumunu   ren
SHOW hba_file;
```

`pg_hba.conf` dosyasının yapısı:

```
# TYPE     DATABASE     USER              ADDRESS         METHOD
local      all          postgres                          trust
host       dvdrental    analist           127.0.0.1/32    md5
host       dvdrental    operator          127.0.0.1/32    md5
host       dvdrental    veri_yoneticisi   127.0.0.1/32    md5
```

Bu yapı ile her kullanıcının yalnızca yetkili IP adreslerinden ve MD5 şifre doğrulaması ile bağlanması zorunlu kılınmaktadır.

3 Veri Şifreleme

MSSQL'deki TDE (Transparent Data Encryption) özelliğinin PostgreSQL karşılığı olarak **pgcrypto** extension'ı kullanılmıştır. pgcrypto ile hassas veriler sütun düzeyinde şifrelenerek yetkisiz erişime karşı korunmaktadır.

3.1 pgcrypto Extension'ının Kurulumu

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;
```

3.2 Hassas Veri Tablosunun Oluşturulması

Müşteri kişisel verileri ve kredi kartı bilgilerini içeren test tablosu oluşturulmuştur:

```
CREATE TABLE musteri_hassas (
    id SERIAL PRIMARY KEY,
    ad VARCHAR(100),
    email TEXT,
    email_sifrelenmis BYTEA,
    kredi_karti TEXT,
    kredi_karti_sifrelenmis BYTEA,
    kayit_tarihi TIMESTAMP DEFAULT NOW()
);
```

3.3 Veri Şifreleme — pgp_sym_encrypt

Hassas veriler `pgp_sym_encrypt` fonksiyonu ile şifrelenmiştir:

```
INSERT INTO musteri_hassas
    (ad, email, email_sifrelenmis, kredi_karti,
       kredi_karti_sifrelenmis)
VALUES
(
    'Ahmet Yilmaz',
    'ahmet@example.com',
```

```
    pgp_sym_encrypt('ahmet@example.com', 'gizli_anahtar_2026'),
    '4111-1111-1111-1111',
    pgp_sym_encrypt('4111-1111-1111-1111', 'gizli_anahtar_2026')
),
(
    'Ayse Kaya',
    'ayse@example.com',
    pgp_sym_encrypt('ayse@example.com', 'gizli_anahtar_2026'),
    '5500-0000-0000-0004',
    pgp_sym_encrypt('5500-0000-0000-0004', 'gizli_anahtar_2026')
);
```

3.4 Şifreli Verinin Görüntülenmesi

Şifreli sütunlar sorgulandığında anlamsız byte dizisi görünmektedir:

```
SELECT id, ad, email_sifrelenmis, kredi_karti_sifrelenmis
FROM musteri_hassas;
```

Bulgu: Şifrelenmiş sütunlar, doğru anahtar olmadan okunamamaktadır. Veritabanına yetkisiz erişim sağlansa bile veriler anlamsız görünmektedir.

3.5 Şifre Çözme — pgp_sym_decrypt

Doğru anahtar ile şifreli veri okunabilmektedir:

```
SELECT id, ad,
    pgp_sym_decrypt(email_sifrelenmis, 'gizli_anahtar_2026') AS
        email_acik,
    pgp_sym_decrypt(kredi_karti_sifrelenmis, 'gizli_anahtar_2026
        ') AS kart_acik
FROM musteri_hassas;
```

3.6 Yanlış Anahtar Testi

Yanlış şifreleme anahtarı ile erişim denendiğinde hata alınmaktadır:

```sql
SELECT pgp_sym_decrypt(email_sifrelenmis, 'yanlis_anahtar')
FROM musteri_hassas
LIMIT 1;
-- SONU : ERROR: Wrong key or corrupt data
```

Sonuç: Yanlış anahtarla şifre çözme girişimi `ERROR: Wrong key or corrupt data` hatası ile engellenmiştir. Bu davranış, TDE ile aynı güvenlik prensibini uygulamaktadır.

4 SQL Injection Testleri

SQL Injection, saldırganların uygulama aracılığıyla veritabanına zararlı SQL kodu göndermesidir. Bu bölümde savunmasız senaryolar simüle edilmiş ve korumalı yöntemler uygulanmıştır.

4.1 Savunmasız Senaryo Simülasyonu

Dinamik SQL kullanımının yarattığı güvenlik açığı gösterilmiştir:

```
-- Her zaman TRUE d nd ren klasik injection
-- Sald rgan giri i: ' OR '1'='1
SELECT * FROM customer
WHERE email = '' OR '1'='1';
-- SONU : TUM musteri kayitlari d ner! Tehlikeli.
```

Bu sorgu, tüm müşteri verilerini yetkisiz kullanıcıya açmaktadır. Gerçek bir uygulamada bu durum ciddi veri ihlali anlamına gelir.

```
-- Injection ile tehlikeli sorgu olu umu      sim lasyon
DO $$
DECLARE
    kullanici_girisi TEXT := ''' OR ''1''=''1';
    sorgu TEXT;
BEGIN
    sorgu := 'SELECT * FROM customer WHERE email = '''
             || kullanici_girisi || '''';
    RAISE NOTICE 'Tehlikeli sorgu: %', sorgu;
END $$;
```

4.2 Korumalı Yöntem — Prepared Statements

Parametreli sorgular (Prepared Statements) ile SQL Injection tamamen engellenmektedir:

```
-- DO RU: Parametreli sorgu      injection'a kapal
```

```sql
PREPARE guvenli_sorgu(TEXT) AS
    SELECT customer_id, first_name, last_name, email
    FROM customer
    WHERE email = $1;

-- Normal kullan m
EXECUTE guvenli_sorgu('mary.smith@sakilacustomer.org');

-- Injection denemesi      artik zararsiz
EXECUTE guvenli_sorgu(''' OR ''1''=''1');
-- SONU : 0 sat r      injection      almad     !

DEALLOCATE guvenli_sorgu;
```

Önemli Fark:

Yöntem	Injection Riski	Sonuç
Dinamik SQL (string birleştirme)	**YÜKSEK**	Tüm veriler açığa çıkar
Prepared Statement	**YOK**	Giriş literal olarak işlenir

Tablo 4: Dinamik SQL ile Prepared Statement güvenlik karşılaştırması

4.3 Ek Güvenlik — VIEW ile Veri Maskeleme

Hassas tablolara direkt erişim kapatılarak yalnızca VIEW üzerinden erişim sağlanmıştır:

```sql
-- Hassas tablolara direkt eri imi kapat
REVOKE ALL ON customer FROM PUBLIC;
REVOKE ALL ON payment FROM PUBLIC;
REVOKE ALL ON staff FROM PUBLIC;

-- Sadece g venli alanlar  i eren VIEW olu tur
CREATE VIEW musteri_gorunum AS
SELECT customer_id, first_name, last_name, active
FROM customer;
-- E-posta ve adres bilgisi dahil edilmedi
```

```sql
-- Analist kullan c s na sadece VIEW eri imi ver
GRANT SELECT ON musteri_gorunum TO analist;
```

Bu yaklaşım ile bir saldırgan VIEW üzerinden erişim sağlasa bile hassas kişisel verilere ulaşamamaktadır.

5 Audit Logları

MSSQL'deki SQL Server Audit özelliğinin PostgreSQL karşılığı olarak **Trigger tabanlı audit log sistemi** oluşturulmuştur. Bu sistem, veritabanındaki her INSERT, UPDATE ve DELETE işlemini otomatik olarak kayıt altına almaktadır.

5.1 Audit Log Tablosunun Oluşturulması

```
CREATE TABLE audit_log (
    log_id        SERIAL PRIMARY KEY,
    kullanici     TEXT       DEFAULT current_user,
    islem_tipi    TEXT,
    tablo_adi     TEXT,
    eski_deger    JSONB,
    yeni_deger    JSONB,
    islem_zamani  TIMESTAMP  DEFAULT NOW(),
    ip_adresi     TEXT       DEFAULT inet_client_addr()::TEXT
);
```

Audit log tablosu şu bilgileri kaydetmektedir:

Sütun	Açıklama
kullanici	İşlemi yapan kullanıcının adı
islem_tipi	INSERT, UPDATE veya DELETE
tablo_adi	İşlemin yapıldığı tablo
eski_deger	Güncellemeden önceki veri (JSON formatında)
yeni_deger	Güncellemeden sonraki veri (JSON formatında)
islem_zamani	İşlemin tam tarihi ve saati
ip_adresi	Bağlanan istemcinin IP adresi

Tablo 5: Audit log tablosu sütun açıklamaları

5.2 Audit Trigger Fonksiyonunun Oluşturulması

```
CREATE OR REPLACE FUNCTION audit_trigger_func()
RETURNS TRIGGER AS $$
```

```
BEGIN
    IF TG_OP = 'INSERT' THEN
        INSERT INTO audit_log(islem_tipi, tablo_adi, yeni_deger)
        VALUES ('INSERT', TG_TABLE_NAME, row_to_json(NEW)::JSONB
           );

    ELSIF TG_OP = 'UPDATE' THEN
        INSERT INTO audit_log(islem_tipi, tablo_adi, eski_deger,
            yeni_deger)
        VALUES ('UPDATE', TG_TABLE_NAME,
                row_to_json(OLD)::JSONB,
                row_to_json(NEW)::JSONB);

    ELSIF TG_OP = 'DELETE' THEN
        INSERT INTO audit_log(islem_tipi, tablo_adi, eski_deger)
        VALUES ('DELETE', TG_TABLE_NAME, row_to_json(OLD)::JSONB
           );
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

5.3 Trigger'ların Tablolara Bağlanması

```
-- customer tablosunu izle
CREATE TRIGGER customer_audit
AFTER INSERT OR UPDATE OR DELETE ON customer
FOR EACH ROW EXECUTE FUNCTION audit_trigger_func();

-- payment tablosunu izle
CREATE TRIGGER payment_audit
AFTER INSERT OR UPDATE OR DELETE ON payment
FOR EACH ROW EXECUTE FUNCTION audit_trigger_func();
```

5.4 Audit Sisteminin Test Edilmesi

```sql
-- Test i lemleri ger ekle tir
UPDATE customer SET active = 0 WHERE customer_id = 1;
UPDATE customer SET active = 1 WHERE customer_id = 1;

INSERT INTO customer
    (store_id, first_name, last_name, email, address_id, active)
VALUES
    (1, 'Test', 'Kullanici', 'test@test.com', 1, 1);
```

5.5 Audit Loglarının Sorgulanması

```sql
SELECT log_id, kullanici, islem_tipi, tablo_adi,
       islem_zamani, yeni_deger
FROM audit_log
ORDER BY islem_zamani DESC;
```

Bulgu: Audit log sistemi sayesinde kimin, ne zaman, hangi tabloda değişiklik yaptığı ve eski/yeni değerlerin ne olduğu tam olarak takip edilebilmektedir. Bu yapı SQL Server Audit'in sağladığı izlenebilirlik seviyesine eşdeğerdir.

5.6 Audit Log Özet Raporu

```sql
--   lem   tiplerine  g re  zet
SELECT islem_tipi,
       tablo_adi,
       COUNT(*) AS islem_sayisi,
       MIN(islem_zamani) AS ilk_islem,
       MAX(islem_zamani) AS son_islem
FROM audit_log
GROUP BY islem_tipi, tablo_adi
ORDER BY islem_sayisi DESC;
```

6 Sonuç ve Değerlendirme

Bu proje kapsamında PostgreSQL veritabanı yönetim sistemi üzerinde kapsamlı güvenlik analizi ve erişim kontrolü çalışmaları gerçekleştirilmiştir.

6.1 Yapılan Çalışmaların Özeti

Güvenlik Konusu	Uygulanan Yöntem	Sonuç
Erişim Yönetimi	RBAC rolleri + pg_hba.conf	3 farklı yetki düzeyi uygulandı
Yetki Kısıtlama	GRANT / REVOKE komutları	Yetkisiz erişim engellendi
Veri Şifreleme	pgcrypto — pgp_sym_encrypt	E-posta ve kart verisi şifrelendi
Yanlış Anahtar Testi	pgp_sym_decrypt — hata testi	ERROR: Wrong key alındı
SQL Injection	Dinamik SQL vs Prepared Statement	Savunmasız ve güvenli senaryo gösterildi
Veri Maskeleme	VIEW ile hassas alan gizleme	Kişisel veriler korundu
Audit Log	Trigger + audit_log tablosu	Tüm değişiklikler kayıt altında

Tablo 6: Yapılan güvenlik çalışmalarının özeti

6.2 Öğrenilen Dersler

- PostgreSQL'de RBAC ile MSSQL Authentication kadar güçlü erişim kontrolü uygulanabilmektedir.
- `pgcrypto` extension'ı, TDE'nin sütun düzeyindeki karşılığı olarak hassas verilerin korunmasını sağlamaktadır.
- SQL Injection saldırıları, Prepared Statement kullanımıyla tamamen engellenebilmektedir. Dinamik SQL kullanımından kaçınılmalıdır.
- VIEW tabanlı veri maskeleme, gereksiz veri açığını minimuma indirmektedir.
- Trigger tabanlı audit sistemi, SQL Server Audit'e eşdeğer bir izlenebilirlik sağlamaktadır.

- `pg_hba.conf` ile IP bazlı erişim kısıtlaması, ağ düzeyinde ek bir güvenlik katmanı oluşturmaktadır.

6.3 Kaynaklar

- PostgreSQL Documentation: `https://www.postgresql.org/docs/`
- pgcrypto Documentation: `https://www.postgresql.org/docs/current/pgcrypto.html`
- PostgreSQL Row Security: `https://www.postgresql.org/docs/current/ddl-rowsecurity.html`
- pgaudit Documentation: `https://github.com/pgaudit/pgaudit`
- OWASP SQL Injection Prevention: `https://owasp.org/www-community/attacks/SQL_Injection`